Smart Notes

Aplicación de Notas Inteligente con IA

Aaron Rodrigo Ramos Reyes
Interacción Humano Computadora
Universidad Autónoma Metropolitana – Unidad Cuajimalpa

25 de marzo de 2026

Índice

1 Introducción **3**

2 Tecnologías Utilizadas **3**

3 Integración con OpenRouter (IA) **3**
3.1 Cómo funciona la integración: 3

4 Pruebas Realizadas con la IA **4**

5 Configuración Manual de Alarmas **6**

6 Heurísticas de Nielsen: Evaluación del Proyecto **7**

7 Áreas de Mejora y Retos Futuros **7**

8 Conclusiones **8**

1. Introducción

Smart Notes es una aplicación móvil desarrollada en Flutter que permite crear, organizar y gestionar notas de forma inteligente.

La característica principal es su **asistente IA integrado**, que analiza automáticamente el contenido de las notas y sugiere acciones útiles como:

- Configurar recordatorios automáticos (alarmas)
- Crear o mover notas a grupos temáticos

La aplicación cuenta con un diseño minimalista, sistema de grupos, arrastrar y soltar, y notificaciones locales.

2. Tecnologías Utilizadas

- **Flutter** + Dart (Framework multiplataforma)
- **Hive** (Base de datos NoSQL ligera y rápida)
- **flutter_local_notifications** (Para recordatorios)
- **OpenRouter** (Servicio de Inteligencia Artificial)
- Material Design 3 (Interfaz moderna y limpia)

3. Integración con OpenRouter (IA)

Para dotar a la aplicación de inteligencia, utilizamos la API de **OpenRouter**, un servicio que permite acceder a múltiples modelos de lenguaje avanzados.

3.1. Cómo funciona la integración:

1. Cuando el usuario guarda una nota, se llama automáticamente al servicio `AIService`.
2. Se envía el título y contenido de la nota junto con la fecha actual al modelo de IA.
3. La IA analiza el texto en lenguaje natural y responde en formato JSON estructurado.
4. El JSON contiene:
   - `has_date`: Si detecta una fecha u hora
   - `date_time`: Fecha y hora en formato YYYY-MM-DD HH:MM
   - `category`: Categoría de la nota
   - `suggested_action`: “alarma”, “crear_grupo” o “ninguna”
   - `explanation`: Explicación breve para el usuario
5. Según la sugerencia, la app muestra un diálogo elegante ofreciendo al usuario aceptar o ignorar la recomendación.

Esta integración permite que la aplicación entienda el contexto de las notas y ayude proactivamente al usuario.

4. Pruebas Realizadas con la IA

Se realizaron las siguientes pruebas para demostrar el funcionamiento del asistente inteligente. En cada caso se muestra una captura de pantalla del resultado.

1. **Reunión con el equipo de Flutter**
   Contenido: "Mañana a las 10 am en la sala de juntas..."
   Resultado: La IA detectó correctamente la fecha y sugirió configurar una alarma para el 26 de marzo a las 10:00.

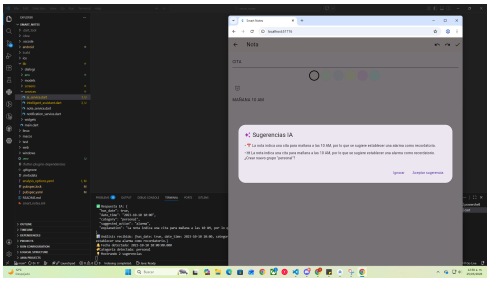

Figura 1: Sugerencia de alarma generada por la IA.

2. **Proyecto Flutter final**
   Contenido: "Terminar la integración de Hive... Fecha de entrega: próxima semana"
   Resultado: La IA sugirió crear un nuevo grupo llamado "trabajo".

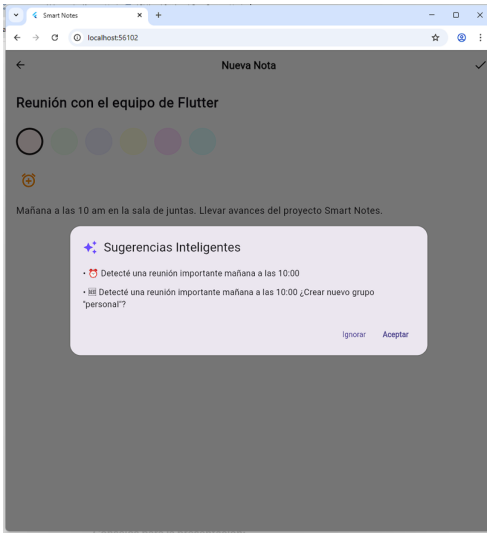

Figura 2: Sugerencia de grupo generada por la IA.

3. **Cita con el dentista**
   Contenido: "El viernes 27 de marzo a las 15:30..."
   Resultado: La IA detectó la cita y ofreció configurar un recordatorio.

4. **Recordar comprar leche**
   Contenido: "Ir al supermercado esta tarde"
   Resultado: La IA sugirió configurar un recordatorio general.

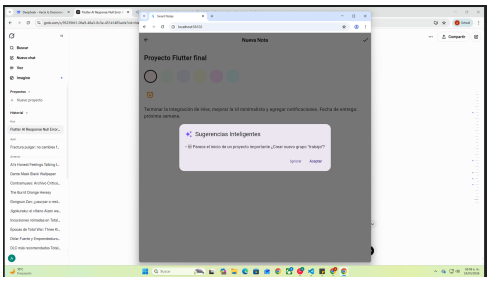

Figura 3: Sugerencia de recordatorio con fecha específica.

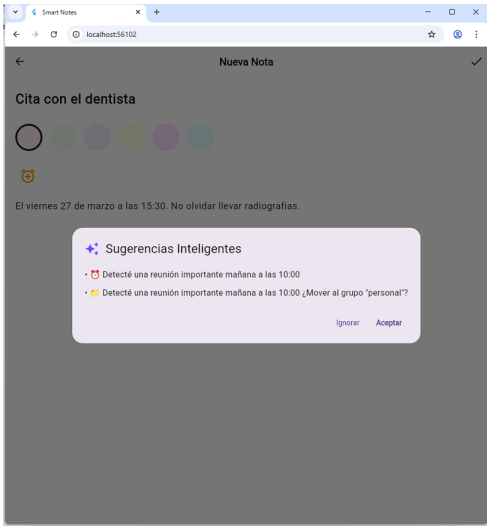

Figura 4: Sugerencia de recordatorio basada en horario.

En todas las pruebas la IA respondió correctamente y las sugerencias fueron aceptadas por el usuario, demostrando la efectividad del sistema.

5. Configuración Manual de Alarmas

Además de las sugerencias automáticas de la IA, la aplicación permite configurar recordatorios de forma manual:

- Al crear o editar una nota, el usuario puede presionar el icono de alarma.
- Se abre un selector de fecha y hora.
- Una vez guardada la nota, se programa una notificación local mediante `flutter_local_notifications`.
- El usuario puede eliminar o modificar el recordatorio en cualquier momento.

Esta funcionalidad asegura que el usuario siempre tenga control total sobre sus recordatorios, incluso si decide no usar las sugerencias de la IA.

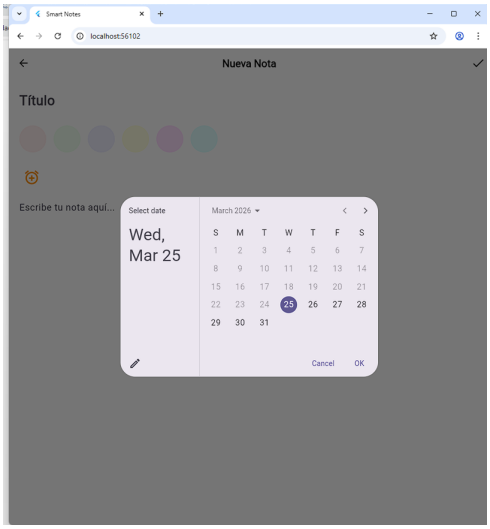

Figura 5: Pantalla de configuración manual de alarma.

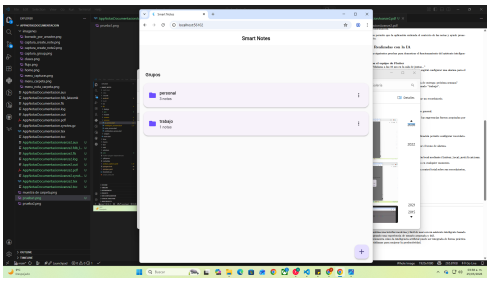

Figura 6: Muestra de carpeta.

6. Heurísticas de Nielsen: Evaluación del Proyecto

A continuación se analiza cómo cumple la aplicación con las 10 heurísticas de usabilidad de Jakob Nielsen:

1. **Visibilidad del estado del sistema:** *Cumple.* La aplicación muestra siempre el estado actual (título en AppBar, lista de notas actualizada en tiempo real mediante `ValueListenableBuilder`).
2. **Relación entre el sistema y el mundo real:** *Cumple.* Usa lenguaje cotidiano ("Nota", "Grupo") e iconos reconocibles.
3. **Control y libertad del usuario:** *Cumple.* Permite deshacer/rehacer, cancelar diálogos y navegar hacia atrás.
4. **Coherencia y estándares:** *Cumple.* Sigue las convenciones de Material Design.
5. **Prevención de errores:** *Cumple parcialmente.* Valida título vacío y pide confirmación antes de eliminar. Falta validar grupos duplicados.
6. **Reconocimiento en lugar de recuerdo:** *Cumple.* Todas las acciones están visibles (menús, iconos, botones).
7. **Flexibilidad y eficiencia de uso:** *Cumple.* Permite arrastrar y soltar para usuarios avanzados, además de menús contextuales.
8. **Diseño estético y minimalista:** *Cumple.* Interfaz limpia, colores pastel, sin elementos superfluos.
9. **Ayuda a los usuarios a diagnosticar errores:** *Cumple parcialmente.* Muestra mensajes claros al fallar la validación, pero no maneja errores de red de la IA.
10. **Ayuda y documentación:** *No cumple.* No existe una pantalla de ayuda o tutorial integrado.

En general, el proyecto cumple con la mayoría de las heurísticas, destacando la usabilidad y la eficiencia en las tareas principales.

7. Áreas de Mejora y Retos Futuros

A pesar del buen funcionamiento actual, se identifican áreas de mejora:

- **Límite de tokens en modelos gratuitos:** Los modelos gratuitos de OpenRouter tienen un límite de tokens por minuto (30 req/min, 60k tokens/min). Esto puede ser insuficiente en producción si la aplicación escala. Como alternativa se podría implementar caché local de respuestas o migrar a un modelo de pago con mayor capacidad.
- **Manejo de errores de red:** Actualmente, si la IA falla por falta de conexión, la app solo muestra un mensaje genérico. Se podría agregar un reintento automático o un modo offline con sugerencias locales.
- **Sincronización en la nube:** La aplicación guarda solo en local. Un futuro desarrollo podría incluir sincronización con servicios en la nube para respaldo y uso multiplataforma.
- **Ampliación de la detección de entidades:** Mejorar el prompt de IA para reconocer horarios más complejos (ej. "cada lunes", "dentro de dos semanas") y sugerir recordatorios recurrentes.
- **Privacidad de los datos:** Asegurar que la API key esté protegida y que los datos de usuario no se compartan con terceros sin consentimiento.

8. Conclusiones

Smart Notes combina una interfaz moderna y fácil de usar con un asistente inteligente basado en OpenRouter, logrando una experiencia de usuario avanzada y útil.

La aplicación demuestra cómo la inteligencia artificial puede ser integrada de forma práctica en herramientas cotidianas para mejorar la productividad, cumpliendo con los principios fundamentales de interacción humano-computadora.